ETF两阶段优选研究框架 - 项目说明书

项目概述

项目名称：传统机器学习与大语言模型融合的ETF优选研究框架
项目性质：学术研究型毕业设计项目
核心目标：在传统量化因子模型预测能力有限的前提下，引入大语言模型进行二次排序，检验是否能够提升投资组合的回测表现与风险收益特征。

文档目录

1. 项目概述
2. 技术架构
3. 项目结构
4. 核心功能模块
5. 实验设计
6. 使用方法
7. 输出结果
8. 核心算法与实现
9. 限制与注意事项
10. 故障排除
11. 附录

技术架构

整体架构图

```
数据层 → 特征工程层 → 传统模型层 → 两阶段策略层 → 回测评估层
```

技术栈

- 编程语言：Python 3.10+
- 核心库：
  - 数据处理：pandas, numpy
  - 机器学习：scikit-learn, lightgbm, xgboost, RandomForest
  - 大语言模型：requests (DeepSeek Chat API)
  - 可视化：matplotlib, seaborn
  - 数据获取：akshare (中国金融数据)
  - 工具库：tqdm (进度条), colorama (终端颜色), tabulate (表格输出)
- 外部服务：DeepSeek API (大语言模型服务)

项目结构

```
etf-main/
├── data/                      # 所有数据文件
│   ├── etf_list.csv         # ETF代码列表（当前100只）
│   ├── etf_data/            # 原始ETF日线数据（akshare下载）
│   ├── factor_data/         # 单ETF因子数据（13个技术因子）
│   ├── all_etf_factors.csv # 合并后的因子总表（约72,000行×26列）
│   └── predictions/         # 模型预测数据
│       ├── test_set_with_predictions.csv  # 测试集预测值
│       └── split_info.json                 # 训练/测试集分割信息
├── scripts/                 # Python脚本
│   ├── download_etf.py      # 下载ETF数据
│   ├── compute_etf_factors.py # 计算量化因子
│   ├── merge_etf_factors.py   # 合并因子数据
│   ├── generate_mock_data.py  # 生成模拟数据（测试用）
│   ├── train_rf_multi_target_backtest.py # 传统模型训练与评估
│   ├── llm_ranking.py       # 大语言模型排序模块（核心）
│   ├── two_stage_backtest.py # 两阶段策略回测对比（核心）
│   ├── run_pipeline.py      # 全流程执行脚本
│   ├── test_api.py          # 测试DeepSeek API连接
│   └── prompt_template.txt # LLM Prompt模板文件
├── results/                 # 所有分析结果文件
│   ├── multi_target/        # 传统模型评估结果
│   │   ├── model_results.txt           # 模型评估指标
│   │   ├── feature_importance_*.png    # 特征重要性图
│   │   └── pred_vs_actual_*.png        # 预测vs实际值图
│   └── two_stage/           # 两阶段策略结果
│       ├── backtest_results.csv         # 调仓详细结果
│       ├── performance_metrics.csv      # 绩效指标对比表
│       ├── performance_report.json      # 详细绩效报告
│       ├── cumulative_returns.png       # 累计收益对比图
│       ├── returns_distribution.png     # 收益分布对比图
│       ├── performance_comparison.png   # 绩效指标对比图
│       └── llm_logs/                    # LLM调用全链路日志
├── docs/                    # 项目文档目录
│   ├── LLM可解释性模块实施计划.md # LLM可解释性功能扩展计划
│   ├── 项目说明书.md           # 本项目说明书
│   └── README.md               # 项目简介
├── requirements.txt        # Python依赖包列表
└── .gitignore             # Git忽略文件配置
```

核心功能模块

1. 数据准备模块

- 数据源：akshare (中国金融数据接口)
- **ETF**样本：100只主流ETF，涵盖宽基指数、行业主题、策略因子、跨境ETF等
- 数据范围：约3年日线数据（最近750个交易日）
- 因子计算：13个技术指标因子（动量、波动率、成交量、移动平均线等）
- 数据生成：支持模拟数据生成（`generate_mock_data.py`）用于测试

2. 传统机器学习模块

- 模型选择：随机森林回归器 (RandomForestRegressor)
- 预测目标：多目标预测（次日收益、5日收益、10日收益）
- 特征工程：标准化处理，排除目标泄漏变量
- 评估指标：R^2、MAE、RMSE、特征重要性分析

3. 大语言模型排序模块

- **API**服务：DeepSeek Chat API
- **Prompt**设计：专业的量化投资分析助手角色
- 约束条件：
  - 禁止联网搜索或使用外部知识
  - 仅做横截面比较，不进行时间序列预测
  - 输出严格的JSON格式
- 因子解释：在Prompt中明确每个因子的经济含义

4. 两阶段策略框架

- 第一阶段：传统机器学习模型排序，选取前50只ETF（可通过参数调整）
- 第二阶段：大语言模型横向比较，重新排序选取前10只ETF（可通过参数调整）
- 对比基准：纯传统模型策略（直接选取前10只ETF）
- 异常处理：LLM API失败时自动回退到传统模型排序
- 配置灵活：支持通过命令行参数调整各阶段选取数量、调仓频率等

5. 回测评估系统

- 调仓频率：每周调仓（可配置为日/周/月）
- 持有期：5个交易日（与5日收益预测目标匹配）
- 权重分配：等权重组合
- 交易成本：暂不考虑
- 绩效指标：
  - 累计收益率、年化收益率
  - 年化波动率、夏普比率
  - 最大回撤、胜率（正收益周期比例）

6. LLM可解释性模块（扩展功能）

- 功能定位：增强策略透明度，让LLM在输出排序结果的同时生成解释性报告
- 成本控制：默认关闭解释功能，按需开启，支持选择性生成（仅对高分ETF生成详细解释）
- 解释内容：每只ETF的多因子分析、优势劣势、风险提示、整体市场分析
- 输出格式：双格式输出 - JSON格式（程序处理） + HTML报告（用户查看）
- 存储结构：结构化存储解释内容，便于检索和分析
- 实施状态：已制定详细实施计划（参见`LLM可解释性模块实施计划.md`），待实现

实验设计

研究假设

1. 传统量化因子模型的预测能力有限（R^2接近或低于0）
2. 大语言模型具备横向比较和综合评估能力
3. 两阶段框架可能提升风险调整后收益

对比策略

- 策略**A**（基准）：仅传统机器学习模型排序 → 选取前10只ETF
- 策略**B**（实验）：传统模型选前50只（可配置） + LLM重新排序 → 选取前10只ETF（可配置）
- 策略**C**（可选）：仅LLM排序（独立测试LLM排序能力）

回测参数

参数	默认值	说明	可配置选项
预测目标	Y_future_5d_return	未来5日收益率	Y_next_day_return, Y_future_5d_return, Y_future_10d_return
调仓频率	W (每周)	每周最后一个交易日调仓	D (每日), W (每周), M (每月)
持有期	5个交易日	与预测目标一致	与预测目标匹配
第一阶段选取	50只	传统模型预选数量	可配置 (15-50)
最终组合	10只	实际持有数量	可配置 (5-20)
ETF样本规模	约20只	实际处理的ETF数量	可通过扩展etf_list.csv增加

参数	默认值	说明	可配置选项
LLM使用	开启	是否使用大语言模型	可关闭或使用模拟模式
权重分配	等权重	组合内ETF权重	等权重
交易成本	不考虑	交易费用和冲击成本	暂未实现

使用方法

环境设置

```
# 1. 克隆项目
git clone git@github.com:MogekoQwQ/etf.git
cd etf

# 2. 创建虚拟环境
python -m venv .venv

# 3. 激活虚拟环境
# Windows: .venv\Scripts\activate
# Linux/Mac: source .venv/bin/activate

# 4. 安装依赖
pip install -r requirements.txt

# 5. 设置DeepSeek API密钥
# Windows
set DEEPSEEK_API_KEY=your_api_key_here
# Linux/Mac
export DEEPSEEK_API_KEY=your_api_key_here
```

快速开始

```
# 全流程执行
cd scripts
python run_pipeline.py

# 跳过数据下载（如果数据已存在）
python run_pipeline.py --skip-download

# 跳过特定步骤
python run_pipeline.py --skip-download --skip-factors --skip-traditional
```

```
# 测试模式（模拟LLM排序，避免API调用）
python run_pipeline.py --mock-llm

# 自定义参数
python run_pipeline.py --target Y_future_10d_return --rebalancing-freq M
--top-n-first 30 --top-n-final 5 --skip-llm

# 常用组合（最小测试）
python run_pipeline.py --skip-download --skip-factors --skip-traditional
--mock-llm --top-n-first 15 --top-n-final 5
```

分步执行

```
# 1. 数据下载（需要akshare网络连接）
python scripts/download_etf.py

# 2. 因子计算
python scripts/compute_etf_factors.py
python scripts/merge_etf_factors.py

# 3. 传统模型训练与评估
python scripts/train_rf_multi_target_backtest.py

# 4. 两阶段回测对比（需要DeepSeek API密钥）
python scripts/two_stage_backtest.py

# 5. 测试API连接
python scripts/test_api.py

# 6. 生成模拟数据（测试用）
python scripts/generate_mock_data.py
```

输出结果

传统模型评估结果（`results/multi_target/`）

- `model_results.txt`：模型评估指标（R^2、MAE、RMSE、特征重要性）
- `feature_importance_*.png`：各目标变量的特征重要性排序图
- `pred_vs_actual_*.png`：各目标变量的预测值与实际值对比图

预测数据文件（`data/predictions/`）

- `test_set_with_predictions.csv`：测试集预测值（核心数据，包含所有目标变量的预测）
- `split_info.json`：训练集/测试集分割信息（日期范围、样本数量）

两阶段策略结果(`results/two_stage/`)

- `backtest_results.csv`: 每次调仓的详细结果（策略A/B选择的ETF、实际收益等）
- `performance_metrics.csv`: 绩效指标对比表（年化收益、波动率、夏普比率等）
- `performance_report.json`: 详细绩效报告（包含参数配置和详细指标）
- `cumulative_returns.png`: 累计收益对比图（策略A vs 策略B）
- `returns_distribution.png`: 收益分布对比直方图
- `performance_comparison.png`: 绩效指标对比柱状图
- `llm_logs/`: 大语言模型调用全链路日志（包含Prompt、响应、解析结果）

核心算法与实现

因子计算算法

```
# 主要技术指标因子
1. 动量因子：momentum_20 = (收盘价 / 20日前收盘价 - 1) * 100
2. 波动率因子：volatility_20 = 过去20日收益率的标准差
3. 成交量因子：volume_mean_20 = 过去20日成交量的均值
4. 移动平均线：MA_5, MA_10 = 5日、10日简单移动平均
5. 当日指标：涨跌幅、振幅、换手率、成交量、成交额
```

两阶段策略算法

```
# 策略A：纯传统模型
def execute_strategy_a(df, date):
    1. 筛选当日数据
    2. 按模型预测值降序排序
    3. 选取前10只ETF

# 策略B：两阶段排序
def execute_strategy_b(df, date):
    1. 第一阶段：按模型预测值排序，选前50只
    2. 第二阶段：调用LLM API进行横向比较排序
    3. 选取LLM排序的前10只ETF
    4. 异常处理：LLM失败时回退到传统模型
```

LLM Prompt设计原理

```
PROMPT_TEMPLATE = """
角色定义：专业的量化投资分析助手
任务：ETF横向比较排序器（非时间序列预测）
约束：
1. 仅基于输入数值，禁止外部知识
2. 不做趋势外推，仅横截面比较
3. 输出严格JSON格式
因子解释：明确每个因子的经济含义
输出格式：{"rankings": [{"code": "xxx", "score": 0.95}, ...]}
"""
```

限制与注意事项

数据限制

1. 样本规模：当前约20只ETF（可通过扩展`etf_list.csv`增加至100只），涵盖主流场内ETF
2. 时间跨度：约3年数据（750个交易日），可能无法覆盖多种市场环境
3. 因子设计：相对基础的技术指标（13个因子），可扩展更多因子类型
4. 数据获取：依赖akshare API稳定性，部分ETF可能数据缺失
5. 数据质量：金融数据存在缺失值、异常值，需要预处理

模型限制

1. 传统模型：随机森林（默认100棵树，深度10）适应大规模数据，但可能不是最优选择，可调整超参数
2. **LLM**处理能力：同时排序50只ETF可能超出LLM有效处理范围（已添加分组比较建议）
3. **LLM**依赖：依赖于DeepSeek API的稳定性、成本和token限制（max_tokens=8000）
4. 预测能力：前提是传统模型预测能力有限（R^2接近0），如模型预测能力强则两阶段改进空间有限
5. 计算资源：处理约72,000行数据需要足够内存和计算时间
6. **LLM**解释性：当前Prompt禁止输出解释，扩展解释功能会增加API成本和复杂度

研究假设

1. **LLM**能力假设：假设LLM具备有效的横向比较和综合评估能力
2. 市场有效性：假设传统因子模型存在改进空间，LLM能提供增量信息
3. 实验性质：重点在方法验证和框架设计，非实盘交易精度
4. 可解释性需求：LLM决策过程需要透明化以增强策略可信度

故障排除

常见问题

1. 导入错误：确保已安装所有依赖 `pip install -r requirements.txt`
2. **API**密钥错误：检查 `DEEPSEEK_API_KEY` 环境变量设置，或在`llm_ranking.py`中直接设置
3. 数据不存在：按顺序运行数据准备步骤，或使用`--skip-download`跳过已有数据
4. 内存不足：减少随机森林树数量或使用数据子集，调整`train_rf_multi_target_backtest.py`中的参数
5. 网络连接问题：akshare数据下载需要稳定网络连接，部分ETF数据可能获取失败
6. **API**调用失败：DeepSeek API有速率限制，建议使用`--mock-llm`测试流程
7. 编码问题：Windows系统可能遇到文件编码问题，确保使用UTF-8编码

调试建议

1. 先使用测试模式：使用 `--mock-llm` 参数测试完整流程，避免API调用
2. 检查日志文件：查看 `results/two_stage/llm_logs/` 中的API调用日志和错误信息
3. 减少**API**调用：减少调仓频率（如改为每月）以降低API调用次数和成本
4. 使用小样本测试：修改 `data/etf_list.csv` 只保留5-10只ETF进行快速验证
5. 分步执行：使用分步执行命令，定位问题发生的具体步骤
6. 检查数据质量：查看`data/all_etf_factors.csv`数据完整性，检查缺失值和异常值
7. 监控资源使用：处理大数据集时监控内存使用，必要时减少数据规模

技术文档

1. akshare文档：金融数据接口使用
2. scikit-learn文档：机器学习模型实现
3. DeepSeek API文档：大语言模型接口

附录

依赖包完整列表

完整依赖列表参见`requirements.txt`文件。主要依赖包分类如下：

核心数据处理

- `pandas` (~2.3.3): 数据表格处理与分析
- `numpy` (~2.2.6): 数值计算库

机器学习与量化分析

- `scikit-learn` (~1.7.2): 机器学习算法库
- `lightgbm` (~4.6.0): 梯度提升决策树
- `xgboost` (~3.1.2): 极限梯度提升库

数据获取

- `akshare` (~1.17.91): 中国金融数据接口

大语言模型集成

- `requests` (~2.32.5): HTTP请求库，用于API调用

可视化

- `matplotlib` (~3.10.7): 绘图库
- `seaborn` (~0.13.2): 统计可视化库

工具库

- `tqdm` (~4.67.1): 进度条显示
- `colorama` (~0.4.6): 终端颜色输出
- `tabulate` (~0.9.0): 表格格式化输出

其他依赖

- 包含70+个间接依赖包，完整列表请查看`requirements.txt`

安装命令：

```
pip install -r requirements.txt
```